HiLabs® | Cleaning dirty data to unlock its hidden potential for healthcare transformation

RosterIQ: Build a Memory-Driven Provider Roster Intelligence Agent

> **Conducted By:** HiLabs at Cognizance 2026 | **Duration:** 36 hours

Submission Link: Will be communicated on the official WhatsApp group and/or registered mail

Link to the dataset: https://drive.google.com/drive/folders/1XJTM7Tq7S5-N8Tq_7Y346CkLqekNGeZZ?usp=share_link

---

🧭 Background

Healthcare payers process millions of provider roster transactions every month - submissions that travel through a complex multi-stage pipeline, namely: ingestion, pre-processing, mapping approval, ISF generation, DART generation, DART UI validation, and finally SPS load. Each roster file contains thousands of individual provider records, each of which can succeed, fail, be skipped, or be outright rejected based on validation and compliance rules.

When pipelines stall or record rejection rates spike, operations teams scramble to diagnose root causes across hundreds of concurrent Roster Operations (ROs) spanning states, lines of business, and source systems. Meanwhile, market-level transaction metrics reveal whether entire regions are falling below acceptable success thresholds - but no one is automatically connecting file-level quality issues to market-level business outcomes.

Your challenge is to build **RosterIQ**, an autonomous AI agent that operates over this two-layered operational data, remembers what it has seen before, reasons across both pipeline performance and record quality, fetches external context

when needed, and surfaces insights through rich visualizations - all to help operations teams act faster and smarter.

🎯 Problem Statement

> **"Design and build an autonomous AI agent that operates on provider roster pipeline and market transaction data (two CSVs), maintains layered memory across interactions, uses tools to diagnose pipeline behavior and record quality issues, and communicates operational insights through dynamic visualizations - all in service of reducing failures and improving roster data quality at scale."**

The agent must be capable of:

- Answering natural language questions about RO pipeline status, stage durations, health flags, and per-file record quality
- Correlating file-level rejection/failure rates with market-level transaction success percentages
- Remembering past queries, learned diagnostic workflows, and domain knowledge about the pipeline
- Using web search to fetch external context (regulatory changes, data standard updates, payer policy shifts)
- Generating charts, health dashboards, trend analyses, and anomaly reports

---

📂 The Datasets

Your agent operates over **two CSV files** that together represent the full picture of roster operations - one at the file/pipeline level, one at the market/transaction level.

📄 CSV 1 - `roster_processing_details.csv` *(File-Level Pipeline & Record Quality)*

Each row represents one Roster Operation (RO) - a roster file submitted by a provider organization, moving through the processing pipeline. It captures both

how the file moved through stages and **how many records inside it succeeded or failed**.

Column	Description
`ID`	Surrogate primary key for the processing record
`RO_ID`	Unique Roster Object identifier for the file/job execution
`RA_FILE_DETAILS_ID`	FK to file metadata (file name, received date, checksum)
`RA_ROSTER_DETAILS_ID`	FK to roster configuration and roster-level details
`RA_PLM_RO_PROF_DATA_ID`	FK to provider/professional master data used during processing
`SRC_SYS`	Source system (e.g., AvailityPDM, Demographic, ProviderGroup)
`ORG_NM`	Provider organization name
`CNT_STATE`	US state where the provider org operates
`LOB`	Line of Business (e.g., Medicare, Medicaid HMO, Commercial PPO/EPO)
`RUN_NO`	Processing iteration (1 = initial run, >1 = reprocessing attempt)
`FILE_STATUS_CD`	Numeric pipeline status (9=Stopped, 45=DART Review, 49=DART Generation, 99=Resolved)
`LATEST_STAGE_NM`	Current pipeline stage name (STOPPED, DART_REVIEW, DART_GENERATION, RESOLVED)
`PRE_PROCESSING_DURATION`	Time (mins) for pre-processing stage
`MAPPING_APROVAL_DURATION`	Time (mins) for mapping approval
`ISF_GEN_DURATION`	Time (mins) for ISF generation
`DART_GEN_DURATION`	Time (mins) for DART generation
`DART_REVIEW_DURATION`	Time (mins) for DART review
`DART_UI_VALIDATION_DURATION`	Time (mins) for DART UI validation
`SPS_LOAD_DURATION`	Time (mins) for SPS load
`AVG_*_DURATION`	Historical average duration per stage (used for benchmarking)

Column	Description
`PRE_PROCESSING_HEALTH` ... `SPS_LOAD_HEALTH`	Per-stage health flag (Green / Yellow / Red); `SPS_LOAD_HEALTH` is derived from `SCS_PCT`
`IS_STUCK`	Boolean - is this RO currently stuck in a stage?
`IS_FAILED`	Boolean - has this RO failed?
`FAILURE_STATUS`	Free-text failure reason (e.g., "Complete Validation Failure")

💡 **Derived Metrics Opportunity:** The raw columns in this dataset are un-aggregated. A strong agent implementation will use its data connector or SQL tool to **derive operational metrics on the fly,** for example: computing a composite pipeline health score from the stage-level health flags, calculating how far each stage's duration deviates from its historical average, estimating retry overhead from `RUN_NO`, or inferring file-level success/failure from `FILE_STATUS_CD` and `LATEST_STAGE_NM`.

📄 CSV 2 - `aggregated_operational_metrics.csv` *(Market-Level Transaction Volume & Success Rates)*

Each row is a monthly aggregate for a state/market, showing how many provider data **transactions** succeeded or failed across all processing iterations for a given client.

Column	Description
`MONTH`	Reporting month (e.g., Jan-26)
`MARKET`	State or regional grouping (e.g., CA, NY, TX, NATIONAL, WNY)
`CLIENT_ID`	Unique identifier of the payer/insurance client
`FIRST_ITER_SCS_CNT`	Transactions successfully processed on the first pass
`FIRST_ITER_FAIL_CNT`	Transactions that failed on the first pass
`NEXT_ITER_SCS_CNT`	Transactions successfully processed after retry/reprocessing
`NEXT_ITER_FAIL_CNT`	Transactions that still failed even after reprocessing
`OVERALL_SCS_CNT`	Total successful transactions across all iterations
`OVERALL_FAIL_CNT`	Total failed transactions across all iterations
`SCS_PERCENT`	Overall transaction success rate (%) for this market and month

Column	Description
`IS_ACTIVE`	Whether this report record is currently active

🧠 Core Requirement: The Memory Architecture

This is the **most heavily weighted component**. Your agent must implement all three memory types with clear, demonstrable behavior during judging.

1. 🗂️ Episodic Memory

"What has this agent investigated before?"

- The agent logs every query, diagnostic action, and finding - timestamped and persistently stored across sessions.
- On return, the agent surfaces relevant history and **detects state changes**: *"Last session you flagged 3 ROs stuck in DART_GENERATION in CA - 2 have since resolved, 1 is still stuck"*
- The agent must track trends over time: if a market's `SCS_PERCENT` was previously flagged as declining, the agent should recognize and continue tracking that thread in future sessions.
- Episodic memory must survive session restarts (use a vector DB, SQLite, or embedded store).

Example episodic triggers:

```
"Have we investigated Norton Hospitals' metrics before?"
"When was the last time KS market dropped below 85% success r
ate?"
"What changed in the pipeline since my last session?"
```

2. ⚙️ Procedural Memory

"How does this agent know how to diagnose things?"

- The agent encodes and reuses diagnostic workflows. If a user defines or refines a process, the agent stores and re-executes it reliably on demand.

- Implement at least **4 named diagnostic procedures:**

Procedure	Description
`triage_stuck_ros`	Finds all IS_STUCK=1 ROs, ranks by time stuck and Red health flags, recommends escalation priority
`market_health_report`	Correlates a market's SCS_PERCENT (CSV 2) with its pipeline-level health flags and stuck RO counts (CSV 1) for the same state
`retry_effectiveness_analysis`	For RUN_NO > 1, derives retry lift rate and determines whether reprocessing is recovering transactions or just adding latency

- Procedures must be **improvable** — if a user refines the logic, the agent updates the stored procedure and confirms what changed.

---

3. 🧬 Semantic Memory

"What does this agent know about the pipeline domain?"

- The agent maintains a knowledge base of domain concepts, including:
  - What each pipeline stage does (pre-processing, ISF generation, DART generation, SPS load, etc.)
  - What `FILE_STATUS_CD` values represent operationally
- What health flag colors mean per stage (Green / Yellow / Red)
- What `IS_STUCK` vs. `IS_FAILED` represent as distinct operational states
- What LOB abbreviations mean (Medicare HMO, Medicaid FFS, Commercial PPO/EPO, etc.)
- What source systems represent (AvailityPDM, Demographic, ProviderGroup)
- How first-iteration vs. retry transaction counts in CSV 2 relate to pipeline reprocessing behavior in CSV 1
- Semantic memory must make reasoning **explainable**: *"I flagged this RO because its DART_GEN_DURATION is 3x its historical average — this typically indicates a data volume spike or a downstream validation bottleneck."*

🛠️ Required Tools

Your agent must have access to and demonstrably use the following tools:

🌐 Web Search Tool

Use web search purposefully in at least **3 demonstrable ways,** for example**:**

- Look up CMS or state Medicaid agency rule changes that may explain rejection spikes in specific markets
- Research what "Complete Validation Failure" means in the context of provider roster compliance standards
- Fetch LOB-specific regulatory context (e.g., Medicaid FFS data submission requirements for KS or VA)
- Look up provider org names (e.g., *"Cedars-Sinai Medical Care Foundation"*, *"MercyOne Medical Group"*) to add business context to pipeline anomalies

📊 Visualization Tools

The agent must generate and render visualisations, for example:

- **Pipeline stage health heatmap** - orgs × stages, colored by health flag (Green/Yellow/Red)
- **Duration anomaly chart** - actual vs. AVG duration per stage, with outlier highlighting
- **Market SCS_PERCENT trend** - monthly success rate by state, with 95% threshold reference line
- **Retry lift chart** - first-iteration vs. post-retry success counts by market, showing how recovery effectiveness
- **Stuck RO tracker** - all IS_STUCK ROs ranked by time-in-stage with health flag overlay

🗃️ Data Query & Analysis Tools

- Filter, sort, aggregate across both CSVs

- Cross-table correlation — join CSV 1 (state-level pipeline health) with CSV 2 (market SCS_PERCENT) for the same state and period
- On-the-fly metric derivation — compute composite scores, duration deviations, retry lift rates, and trend deltas using the agent's data connector or SQL tool
- Anomaly detection — flag ROs where any stage duration exceeds a configurable multiple of its historical average

📝 Operational Report Generator

- On demand: produce a structured **Pipeline Health Report** for a given state, org, or time window
- Include: summary statistics, flagged ROs, stage bottlenecks, derived health metrics, market SCS_PERCENT context, and recommended actions

---

🏆 Judging Criteria

Criterion	Weight	What Judges Look For
Memory Architecture Depth	35%	All 3 types implemented, persistent, demonstrably triggered during interactions
Tool Use & Web Integration	25%	Web search used purposefully with pipeline/market context; tools compose naturally in multi-step reasoning
Data Visualization Quality	20%	Charts are accurate, contextually triggered, and actionable - not decorative
End-to-End Agent Reliability	20%	Agent handles multi-step, cross-table queries without breaking; graceful error handling

---

🚀 Bonus Challenges *(for extra points)*

- **Proactive monitoring:** Agent scans both CSVs on a schedule and pushes alerts for new stuck ROs or markets trending toward SCS_PERCENT thresholds — without being asked.

- **Multi-agent architecture:** A Supervisor agent delegates to specialist sub-agents — a Pipeline Health Agent and a Market Analytics Agent — that collaborate to answer cross-domain queries.
- **Root cause chaining:** Agent automatically traces a market's low SCS_PERCENT back through pipeline stages to specific orgs, source systems, or LOBs — generating a full causal chain with evidence.

📦 Deliverables

Teams must submit:

1. **Working demo** - Live or recorded walkthrough covering at least 5 interaction types, spanning both CSVs and all 3 memory types
2. **Architecture diagram** - Memory stores, tool integrations, data flow, and agent reasoning loop clearly illustrated
3. **GitHub repository** - Documented code with `README.md` covering setup, design decisions, and memory architecture explanation
4. **3-minute pitch** - Focus on: *"How does your memory system make this agent smarter over time?"*

🧰 Suggested Tech Stack *(not prescriptive)*

Component	Options
Agent Framework	LangChain, LlamaIndex, AutoGen, CrewAI, custom loop
LLM	See free-tier options below ⬇️
Episodic Store	ChromaDB, Pinecone, Weaviate, SQLite + embeddings
Procedural Store	YAML/JSON workflow registry, vector-tagged function store
Semantic Store	Embedded domain ontology, FAISS, custom knowledge graph
Data Layer	Pandas, DuckDB, SQLite (for CSV querying and cross-table joins)
Visualization	Plotly, Matplotlib, Seaborn, Streamlit, Grafana
Web Search	Tavily, SerpAPI, Brave Search API
Frontend	Streamlit, Gradio, React, or CLI

💡 Recommended Free-Tier LLM Options

All three options below have free tiers with no billing required for basic usage

1. Google AI Studio (Gemini API)

- Portal: aistudio.google.com
- Free tier: No-cost UI + API access with capped RPM/TPM; no billing needed for basic usage
- Example endpoint:

```
<https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent>
```

2. OpenRouter (Multiple Free Models)

- Site: openrouter.ai
- Free models include: `google/gemini-2.0-flash-exp:free`, `mistralai/devstral-2512:free`, `qwen/qwen3-coder-32b:free`, and others
- Example endpoint:

```
<https://openrouter.ai/api/v1/chat/completions>
```

3. Mistral AI - La Plateforme

- Site: mistral.ai / console.mistral.ai
- Free tier for experimentation on models like `mistral-small-latest`
- Example endpoint:

```
<https://api.mistral.ai/v1/chat/completions>
```

⚖️ Constraints & Rules

- The provided dataset is **proprietary operational data** - it must not be published, shared publicly, or committed to public repositories (use private

GitHub repos for submission)

- **Never hardcode API keys or credentials** in your code - use environment variables or a `.env` file, and ensure `.env` is listed in `.gitignore`
- Web search must be used **at least 3 demonstrable times** with outputs visibly influencing agent reasoning
- All 3 memory types must be **independently demonstrated** and shown working **in combination** at least once
- Cross-table analysis (correlating both CSVs) must be demonstrated **at least twice** during judging

---

📋 Submission Requirements

Every team must submit all three of the following:

1. 📄 Documentation PDF

A well-structured PDF covering:

- **Problem approach** - how your team interpreted the problem and scoped your solution
- **Architecture & schematics** - diagrams of your agent's memory architecture, tool integrations, and data flow
- **Memory design decisions** - how and why you implemented each of the three memory types the way you did
- **Innovations** - any novel techniques, design patterns, or creative extensions beyond the core requirements
- **Challenges & learnings** - what didn't work, what you'd do differently, and what surprised you

2. 💻 GitHub Repository *(private, with access granted to judges)*

- A clean, well-organized codebase
- A `README.md` covering: project overview, setup instructions, environment variables needed, and how to run the agent

- A `.env.example` file showing required keys (never commit real keys)
- Both CSVs included or a script to generate/load them

3. 🎥 Demo Video *(2–3 minutes, screen recording)*

A screen-recorded walkthrough that:

- Briefly explains the solution architecture (30–45 seconds)
- Demonstrates the agent handling at least **3 distinct interaction types** live
- Shows all **3 memory types** being triggered visibly during the demo
- Highlights at least **one visualization** and **one web search** in action

> Upload the video to YouTube (unlisted), Google Drive, or Loom and include the link in your `README.md` and PDF.